Test Strategy

QADemo E-Commerce Application

Document Type:	Test Strategy
Project:	QADemo E-Commerce Application
Testing Type:	Manual Web Application Testing
Testing Approach:	Risk-Based Testing
Primary Scope:	Customer-facing e-commerce functionality
Application:	QADemo
Test Environment:	Web Application
Status:	Final
Version:	1.0
Prepared By:	Humayra Siddika Maharun

Purpose

The purpose of this Test Strategy is to define the overall testing approach for the **QADemo E-Commerce Application**.
This document establishes:

- What will be tested
- What will not be tested
- How testing will be performed
- Which testing techniques will be used
- How risks will influence test priority
- Test environment and test data requirements
- Entry and exit criteria
- Defect management and regression approach
- Test deliverables and reporting

The strategy is designed to provide structured, traceable and risk-focused testing throughout the project lifecycle.

Test Objectives

The main objectives of testing are to:

1. Verify that customer-facing functionality behaves according to the approved requirements.
2. Validate the finalized User Stories and Acceptance Criteria.
3. Identify functional defects before release.
4. Verify positive, negative, boundary and error-handling behavior.
5. Validate important business flows from the customer's perspective.
6. Verify that changes and defect fixes do not introduce regressions.
7. Validate API behavior for applicable application functionality.
8. Perform database validation where applicable using SQL.
9. Evaluate browser compatibility and responsive behavior.
10. Identify usability and accessibility issues that may affect customers.
11. Maintain traceability between Requirements, User Stories, Acceptance Criteria, Test Scenarios, Test Cases and Defects.
12. Produce objective test evidence and a final test summary.

Scope

In Scope

Testing will focus on the **customer-facing e-commerce functionality** defined in the approved requirements and finalized User Stories. The scope includes the applicable areas represented in the project requirements, User Stories and Acceptance Criteria,
including:

- Customer authentication and login
- Invalid login and authentication handling
- Product browsing
- Product-related customer interactions
- Shopping cart
- Cart item management
- Checkout and applicable order flow
- Customer order-related functionality
- Error handling and validation
- Business rules defined in the requirements
- Customer-facing UI behavior
- Responsive behavior
- Browser compatibility
- Applicable API functionality
- Applicable database validation

The exact test coverage will be derived from the finalized:

- Business Requirements
- User Stories
- Acceptance Criteria

These artifacts will be treated as the requirements baseline for test design.

Out of Scope

The following areas are outside the current project scope:

- Customer registration
- Admin functionality
- Admin-side workflows
- Admin dashboard testing
- Admin product management
- Admin order management
- Admin user management
- Features not defined in the approved project requirements

Out-of-scope functionality will not be included in the project's test coverage or release assessment.

Test Approach

Testing will follow a **risk-based, requirements-driven approach**.
The testing process will follow this general flow:

Requirements → User Stories → Acceptance Criteria → Risk Analysis → Test Scenarios → Test Cases → Test Data → Execution → Defects → Retest → Regression → Test Summary

Testing priority will be determined by:

- Business impact
- Functional criticality
- Likelihood of failure
- Customer impact
- Security or data implications
- Dependency on other functionality
- Complexity of the workflow
- Defect history

High-risk and business-critical functionality will receive greater testing depth and earlier execution.

Testing Levels and Types

Functional Testing

Functional testing will verify that the application performs according to the approved requirements and Acceptance Criteria.
Coverage will include:

- Valid input
- Invalid input
- Required fields
- Business rules
- Expected system responses
- Error messages
- State changes
- Data persistence
- Customer workflows

Smoke Testing

Smoke testing will be performed after a new build or major deployment to determine whether the application is stable enough for detailed testing.
The smoke test will cover critical functions such as:

- Application availability
- Login
- Product access
- Product selection
- Cart
- Checkout
- Critical customer workflow

If critical smoke tests fail, detailed testing may be paused until the build becomes testable.

Sanity Testing

Sanity testing will be performed after targeted changes or fixes to verify that the affected functionality works as expected before broader regression testing.

Regression Testing

Regression testing will verify that new changes or defect fixes have not negatively affected existing functionality.
Regression scope will be determined based on:

- Changed functionality
- Related functionality
- Defect severity
- Dependencies
- Risk level

Regression testing will focus particularly on critical customer journeys and areas with previous defects.

Retesting

Retesting will be performed after a defect has been fixed.
The original failed test case will be executed again to verify that the reported defect has been resolved. Retesting will be distinguished from regression testing:

Retesting = Verify the specific defect fix
Regression Testing = Verify that the fix did not break other functionality

Exploratory Testing

Exploratory testing will complement scripted testing.
It will be used to identify issues that may not be explicitly covered by predefined test cases.
Exploratory testing may focus on:

- Unusual user behavior
- Unexpected input combinations
- Navigation behavior
- Boundary conditions
- UI inconsistencies
- Workflow interruptions
- Error recovery
- Usability issues

Exploratory sessions will be documented with the area tested, test idea, observations and discovered defects.

API Testing

Applicable APIs will be tested using **rustrest** (link).
API testing will verify:

- HTTP methods
- Request parameters
- Request body
- Response status codes
- Response structure
- Response data
- Required fields
- Invalid requests
- Error handling
- Data consistency where applicable

API testing will complement UI testing rather than replace it.

Database Testing

Database validation will be performed using SQL where applicable.
The purpose is to verify:

- Data persistence
- Data accuracy
- Data consistency
- Expected record creation or modification
- Relationships between relevant data
- Data integrity after customer actions

Database validation will be performed only for functionality where database-level verification is relevant and accessible.

Compatibility Testing

Compatibility testing will verify application behavior across supported browsers.
The primary browser set will include:

- Google Chrome
- Mozilla Firefox
- Microsoft Edge

Testing will focus on:

- Functional behavior
- Layout
- Navigation
- Forms
- Buttons and controls
- Browser-specific UI issues
- JavaScript-related behavior

Responsive Testing

Responsive testing will evaluate customer-facing behavior across different screen sizes.
The test coverage will include representative:

- Desktop resolution
- Laptop resolution
- Tablet resolution
- Mobile resolution

The objective is to identify:

- Layout problems
- Overlapping elements
- Hidden content
- Incorrect scaling
- Horizontal scrolling
- Unusable controls
- Navigation problems
- Text or image display issues

Usability Testing

Usability testing will evaluate whether customers can understand and use the application effectively. The testing will focus on:

- Clear navigation
- Understandable labels
- Consistent UI behavior
- Clear validation messages
- Useful error messages
- Logical customer workflows
- Visibility of important actions

Usability findings will be documented separately from functional failures where appropriate.

Accessibility Checks

Basic accessibility checks will be included for customer-facing interfaces.
The checks may include:

- Keyboard navigation
- Visible focus
- Form labels
- Meaningful button/link text
- Text readability
- Basic color/contrast observations
- Accessible error messaging
- Alternative text where applicable

Risk-Based Test Prioritization

Testing priority will be determined through risk analysis.

Risk Level	Testing Priority	Approach
High	Highest	Extensive functional, negative, boundary, integration, regression and exploratory coverage
Medium	Normal	Functional, negative and relevant compatibility/regression coverage
Low	Lower	Core functional and basic validation coverage

High-risk areas will be tested earlier and with greater depth.

Examples of high-impact customer flows include:

- Authentication
- Product selection
- Cart operations
- Checkout
- Order-related functionality
- Payment-related functionality

Risk priority may be revised if new defects, requirements or changes are identified during testing.

Test Design Techniques

The following test design techniques will be used where applicable:

Equivalence Partitioning
Inputs will be divided into valid and invalid groups so representative values can be tested efficiently.

Boundary Value Analysis
Boundary values and values immediately around boundaries will be tested where requirements define limits.

Decision Table Testing
Decision tables will be used when system behavior depends on multiple business conditions.

State Transition Testing
State-based behavior will be tested where an action changes the state of an entity or workflow.

Error Guessing
Tester experience will be used to identify likely failure conditions that may not be explicitly described in requirements.

Scenario-Based Testing
End-to-end customer scenarios will be created to validate realistic business workflows.

Test Environment

Application: QADemo E-Commerce Application

Application URL: https://qademo.com/

Browsers:
Primary browsers:

- Google Chrome
- Mozilla Firefox
- Microsoft Edge

Browser versions will be recorded during execution where relevant.

Responsive Devices / Resolutions:
Representative screen sizes will be used for:

- Desktop
- Laptop
- Tablet
- Mobile

API Testing Environment: *'rustrest (link)'* will be used for applicable API testing.

Database Testing Environment:
SQL will be used for applicable database validation. Database access and available data will determine the depth of database-level verification.

Test Data Strategy

Test data will be prepared based on the requirements and test scenarios.
The data set will include, where applicable:

- Valid customer credentials
- Invalid credentials
- Locked-user credentials
- Valid product data
- Invalid input
- Boundary values
- Empty values
- Special characters
- Different quantities
- Different combinations of cart items
- Checkout-related data
- API request data

The following project accounts are available for testing:

Account	Purpose
standard_user / standard123	Standard customer testing
locked_user / locked123	Locked-user / authentication behavior testing

Admin credentials may exist in the test environment but admin functionality is outside the approved project scope and will not be included in the test coverage.

Test Case Priority

Test cases will be prioritized according to business and technical risk.

Critical
Tests essential customer journeys or functionality where failure could prevent normal application use.

High
Tests important business functionality where failure could significantly affect customers.

Medium
Tests normal functionality where failure has limited or recoverable impact.

Low
Tests with comparatively limited customer or business impact.

Priority may be changed during execution based on discovered defects and risk.

Entry Criteria

Testing may begin when:

- Approved requirements are available.
- Finalized User Stories are available.
- Acceptance Criteria are available.
- Test environment is accessible.
- Required test accounts are available.
- Test data is prepared.
- Test scenarios and/or test cases are ready for the applicable testing phase.
- Major blocking environment issues have been resolved.

For regression testing, the relevant defect fix or application change must be available for verification.

Exit Criteria

Testing for a defined testing cycle may be considered complete when:

- Planned test execution for the applicable scope is completed.
- Critical customer workflows have been tested.
- High-risk functionality has received appropriate coverage.
- Critical and high-severity defects have been addressed or formally documented with an agreed disposition.
- Required defect retesting has been completed.
- Relevant regression testing has been completed.
- Test results have been documented.
- Remaining known risks are recorded.
- Test summary and release observations have been prepared.

Defect Management

Defects will be documented with sufficient information for reproduction and investigation.
Each defect should contain:

- Defect ID
- Summary
- Module / Feature
- Environment
- Preconditions
- Steps to Reproduce
- Test Data
- Expected Result
- Actual Result
- Severity
- Priority
- Evidence
- Status
- Related Test Case
- Related User Story / Requirement

Typical defect lifecycle:

New → Assigned → In Progress → Fixed → Retest → Closed

If the defect persists:

Retest → Reopened

Defects that cannot be immediately resolved may be documented with an appropriate status and included in the final risk assessment.

Priority and Severity

Severity and priority will be treated as separate concepts.

Priority
Priority represents how urgently the defect should be addressed.
Priority will be determined using business impact, risk, customer impact and release requirements.

Severity

Severity represents the impact of the defect on the application or customer.

Severity	Description
Critical	Causes a major system/business failure or blocks a critical customer workflow
High	Significant functional impact with substantial customer/business effect
Medium	Functional issue with limited or recoverable impact
Low	Minor issue with limited impact

Traceability

Requirements Traceability will be maintained throughout the project.
The traceability chain will be:

Business Requirement → User Story → Acceptance Criteria → Test Scenario → Test Case → Execution Result → Defect

The **Requirements Traceability Matrix (RTM)** will be used to identify:

- Covered requirements
- Missing test coverage
- Test execution status
- Failed tests
- Related defects

The finalized User Stories and Acceptance Criteria will be treated as the baseline for test design.

Test Deliverables

The project will produce the following QA artifacts:

1. Requirements / Project Notes
2. Business Requirements
3. Requirement Analysis
4. User Stories
5. Acceptance Criteria
6. Test Strategy
7. Risk Analysis
8. Test Scenarios
9. Test Cases
10. Test Data
11. Requirements Traceability Matrix
12. Test Execution Results
13. Defect Reports
14. Retest Results
15. Regression Results
16. API Test Results
17. SQL Validation Results
18. Exploratory Testing Notes
19. Compatibility / Responsive Testing Results
20. Test Summary Report
21. Release Recommendation based on documented test evidence and remaining risks

Metrics and Reporting

The following metrics may be collected during execution:

- Total test cases
- Executed test cases
- Passed test cases
- Failed test cases
- Blocked test cases
- Not executed test cases
- Pass percentage
- Defect count
- Defects by severity
- Defects by priority
- Defect status
- Retest results
- Regression results
- Requirement coverage
- Test case coverage

Regression Strategy

Regression testing will be risk-based. A full regression suite will not necessarily be executed after every minor change.
Regression scope will be selected based on:

1. Changed functionality
2. Dependencies
3. Defect severity
4. Affected customer workflows
5. Historical defect areas
6. Risk level

Critical customer journeys will remain part of the regression baseline.

Release Assessment

At the end of testing, the overall test result will be summarized using objective evidence.
The assessment will consider:

- Requirement coverage
- Critical workflow coverage
- Test execution results
- Open defects
- Defect severity
- Defect trends
- Regression results
- Known limitations
- Remaining product risks

The final release assessment will document the evidence and known risks rather than relying on a single test metric.

Roles and Responsibilities

As this is an individual QA portfolio project, the tester will perform the following responsibilities:

- Analyze requirements
- Review User Stories and Acceptance Criteria
- Identify testing risks
- Design test scenarios
- Design and execute test cases
- Prepare test data
- Perform functional testing
- Perform exploratory testing
- Perform API testing
- Perform SQL validation
- Perform compatibility testing
- Perform responsive checks
- Report and manage defects
- Perform retesting
- Perform regression testing
- Maintain the RTM
- Analyze test results
- Prepare the final Test Summary Report

Assumptions and Constraints

Assumptions

- The finalized requirements represent the approved project scope.
- The finalized User Stories and Acceptance Criteria are the testing baseline.
- The application remains accessible during the testing period.
- Required test accounts remain available.
- Applicable API and database access is available for the planned validation activities.

Constraints

- Testing is performed within the available project environment.
- Browser and device coverage is representative rather than exhaustive.
- Database and API coverage depends on available access and application functionality.
- Registration and admin functionality are excluded from this project.
- Testing will focus on customer-facing functionality.
- Test coverage will be driven by risk and actual scope rather than an arbitrary number of test cases.

Overall Testing Workflow

The project testing workflow will follow:

1. Requirement Review
2. User Story & Acceptance Criteria Review
3. Risk Analysis
4. Test Strategy
5. Test Scenario Design
6. Test Case Design
7. Test Data Preparation
8. RTM Preparation
9. Test Execution
10. Defect Reporting
11. Retesting
12. Regression Testing
13. Additional Testing — API / SQL / Exploratory / Compatibility / Accessibility
14. Test Metrics & Analysis
15. Test Summary Report

Strategy Summary

The QADemo E-Commerce Application will be tested using a **requirements-driven and risk-based strategy** focused on customer-facing functionality.

Functional testing will form the core of the strategy, supported by exploratory testing, API testing, SQL validation, compatibility testing, responsive testing, usability checks and basic accessibility checks.

Testing priority will be determined by business impact and risk. Critical customer journeys will receive greater depth and will form the core of the regression strategy.

The finalized **User Stories and Acceptance Criteria** will serve as the baseline for test design and traceability.

The project will evaluate product quality based on **coverage, execution results, defect status, regression results and remaining risks**, rather than an arbitrary target number of test cases.

The final Test Summary will provide documented evidence of what was tested, what was found, what was fixed or remains open, along with what risks remain at the end of the testing cycle.